

Architecture Overview Report

Project: HobbyFi Vendor Portal AI Copilot

1. Tools & Frameworks Chosen

Category	Technology	Version	Purpose
API Framework	FastAPI	0.115+	Async HTTP, validation, OpenAPI, SSE streaming
Agent Runtime	CrewAI	0.80+	Multi-agent orchestration (4-agent sequential pipeline)
LLM Orchestration	LangChain + LangChain Core	0.3+	LLM abstraction, prompt templates, structured output
LLM Provider	Google Gemini	3.1 Flash-Lite	Low-latency, tool-capable, structured JSON output
LLM Client	langchain-google-genai	2.0+	LangChain integration for Gemini
Database (Async)	SQLAlchemy 2.0 + asyncpg	2.0.36+ / 0.29+	Async ORM, PostgreSQL driver
Database (Sync)	SQLAlchemy + psycopg2	2.0.36+	Sync sessions for CrewAI tools
Local DB	aiosqlite	0.20+	SQLite for development/testing
Validation	Pydantic v2	2.9+	Schema validation, settings management
Streaming	sse-starlette	2.1+	Server-Sent Events for chat streaming
Testing	pytest + pytest-asyncio	8.3+ / 0.24+	Async test support
Containerization	Docker + gunicorn	22+	Production deployment
Observability	Structured JSON logging	stdlib	Correlation IDs, phase timers, rate-limit headers

What We're NOT Using (and Why)

Framework	Reason for Not Using
Mastra	Not adopted – CrewAI provides the sequential pipeline pattern we need with built-in tool calling and agent roles
LangGraph	Overkill for our deterministic 4-agent sequential flow; CrewAI's <code>Process.sequential</code> is simpler
LlamaIndex	No RAG/knowledge-base needs – our tools query live SQL data directly
AutoGen	More suited for conversational multi-agent; we need a fixed pipeline
Haystack	Designed for document search pipelines, not SQL-backed agent tools

2. Memory Strategy

Current Implementation (Phases 0–3)

Memory Type	Implementation	Scope
Short-term (Conversation)	In-DB <code>chat_messages</code> table	Per-session, loaded on each turn (last 20 messages)
Working Memory (Agent Turn)	<code>ToolRunContext</code> (in-memory)	Per-agent-turn; captures tool traces + UI blocks
Long-term (Domain Knowledge)	Not implemented	—
User Preferences	Not implemented	—

Data Model

```
# chat_sessions: id, vendor_id, vendor_user_id, created_at
# chat_messages: id, session_id, role, content(JSON), created_at
```

- **Content structure:** `{"text": "...", "blocks": [...], "tool_traces": [...]}`
- **Context window:** Last 20 messages loaded per turn (configurable)
- **No vector embeddings** – all context is structured JSON + SQL

Future Memory Enhancements (Phase 4+)

Feature	Approach
---------	----------

Feature	Approach
Semantic memory	pgvector + embeddings for past Q&A retrieval
User preference learning	Extract preferences from tool calls → store in <code>vendor_users.preferences</code>
Cross-session context	Summarize older sessions → inject as system prompt
Tool call caching	Cache frequent read tool results (e.g., <code>list_games</code>) with TTL

3. Guardrails Framework

Current Guardrails (Implemented)

Layer	Mechanism	Enforcement Point
Input Validation	Pydantic schemas on all tool args	Tool <code>_run()</code> entry
RBAC Gates	<code>check_tool_permission(tool_name, role)</code>	Every tool <code>_run()</code> start
Tenancy Isolation	<code>vendor_id</code> from auth context only	All repositories filter <code>WHERE vendor_id = :ctx_vendor_id</code>
Write Safety	Propose tools only create proposals – never mutate	<code>propose_*</code> tools insert <code>action_proposals</code> (status=pending)
Execution Gate	Human decision required on <code>/proposals/{id}/decide</code>	FastAPI route (not LLM)
Idempotency	Unique <code>idempotency_key</code> per proposal	DB unique constraint
Expiry	<code>expires_at</code> on proposals (default 30 min)	Background task + <code>/decide</code> validation
Audit Trail	Immutable <code>audit_events</code> for every tool call, proposal, decision	Auto-inserted in tools + services
Cross-vendor	Returns 404 (not 403) to avoid enumeration	Repositories + FastAPI deps
Rate Limiting	30 req/min, 200 req/hr per <code>vendor_user</code>	Middleware + headers

Guardrails NOT Yet Implemented

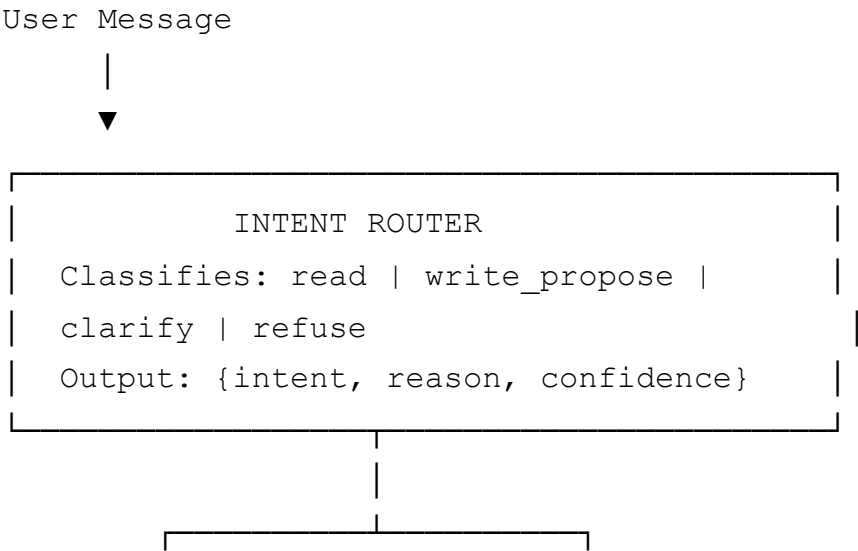
Guardrail	Status	Planned
Output validation	✗	Phase 4 – JSON schema validation on Composer output
PII redaction	✗	Phase 4 – Log sanitization for emails/names
Prompt injection detection	✗	Phase 4 – Heuristic + LLM-based classifier
Tool call budget	✗	Phase 4 – Max N tool calls per turn
Content safety (Gemini)	⚠ Built-in only	Use Gemini safety settings

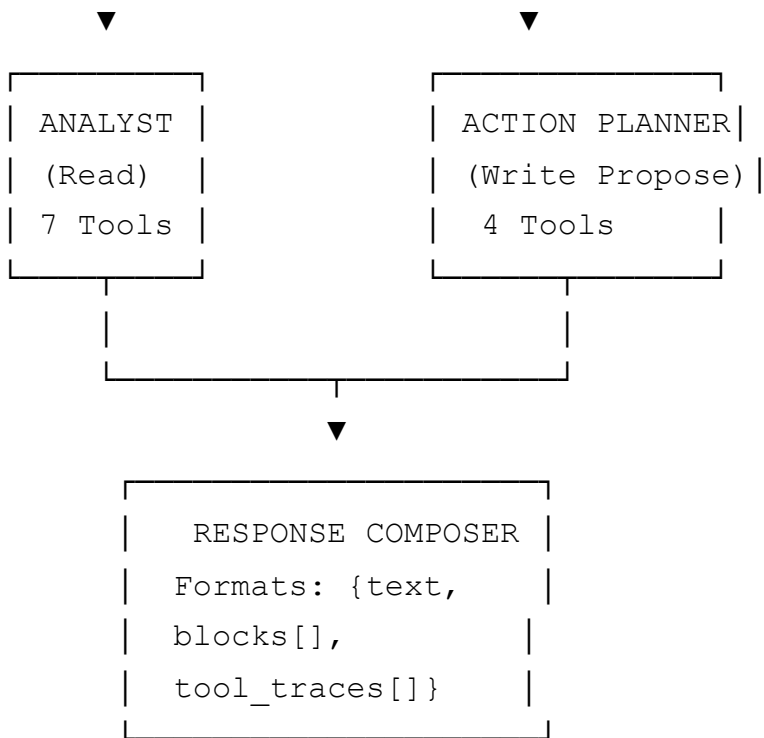
RBAC Matrix

Tool Category	Tool	viewer	support	admin	owner
Read (7)	list_games, list_trial_users, get_revenue, search_users, get_user, get_membership, get_vendor_summary	✓	✓	✓	✓
Propose (4)	propose_extend_trial, propose_change_plan, propose_suspend_user, propose_update_membership_dates	✗	✓	✓	✓
Approve	POST /proposals/{id}/decide	✗	✓	✓	✓

4. Workflow Orchestration

4-Agent Sequential Pipeline (CrewAI)





CrewAI Configuration

```
# app/agent/crew.py
Crew(
    agents=[router, analyst, planner, composer],
    tasks=[route_task, analyze_task, plan_task, compose_task],
    process=Process.sequential, # Fixed order, no loops
    verbose=False,
    memory=False, # We manage memory via DB + ToolRunContext
)
```

Tool Execution Flow

Step	Component	Details
1	FastAPI route	Auth → VendorContext → Rate limit
2	ChatService	Load history, build tools via <code>build_read_tools()</code> / <code>build_write_tools()</code>
3	CrewAI	Sequential agent execution
4	Tools	Each tool: RBAC → repo query → record trace → return JSON
5	ToolRunContext	Aggregates traces + UI blocks (tables, KPIs, proposal cards)
6	Composer	Receives specialist output → formats final response

Step	Component	Details
7	ChatService	Persist messages + audit event
8	Return	ChatTurnResponse or SSE stream

Special Flows

Read Query Flow

Router → "read" intent

- Analyst invokes 1-N read tools (vendor-scoped)
- Composer formats {text, blocks[], tool_traces[]}

Write Proposal Flow

Router → "write_propose" intent

- Planner invokes 1 propose_* tool
- Tool builds {before, after} preview
- INSERT action_proposal (pending, expires_at)
- INSERT audit_event (proposal.create)
- Returns {proposal_id, preview, "awaiting approval"}
- UI shows Approval Card
- Human clicks Approve → POST /decide
- ApprovalService: re-validate → execute mutation → audit

Clarification Flow

Router → "clarify" intent

- Planner not invoked
- Composer asks for missing params

Refusal Flow

Router → "refuse" intent (cross-vendor, SQL injection, bulk delete, secrets)

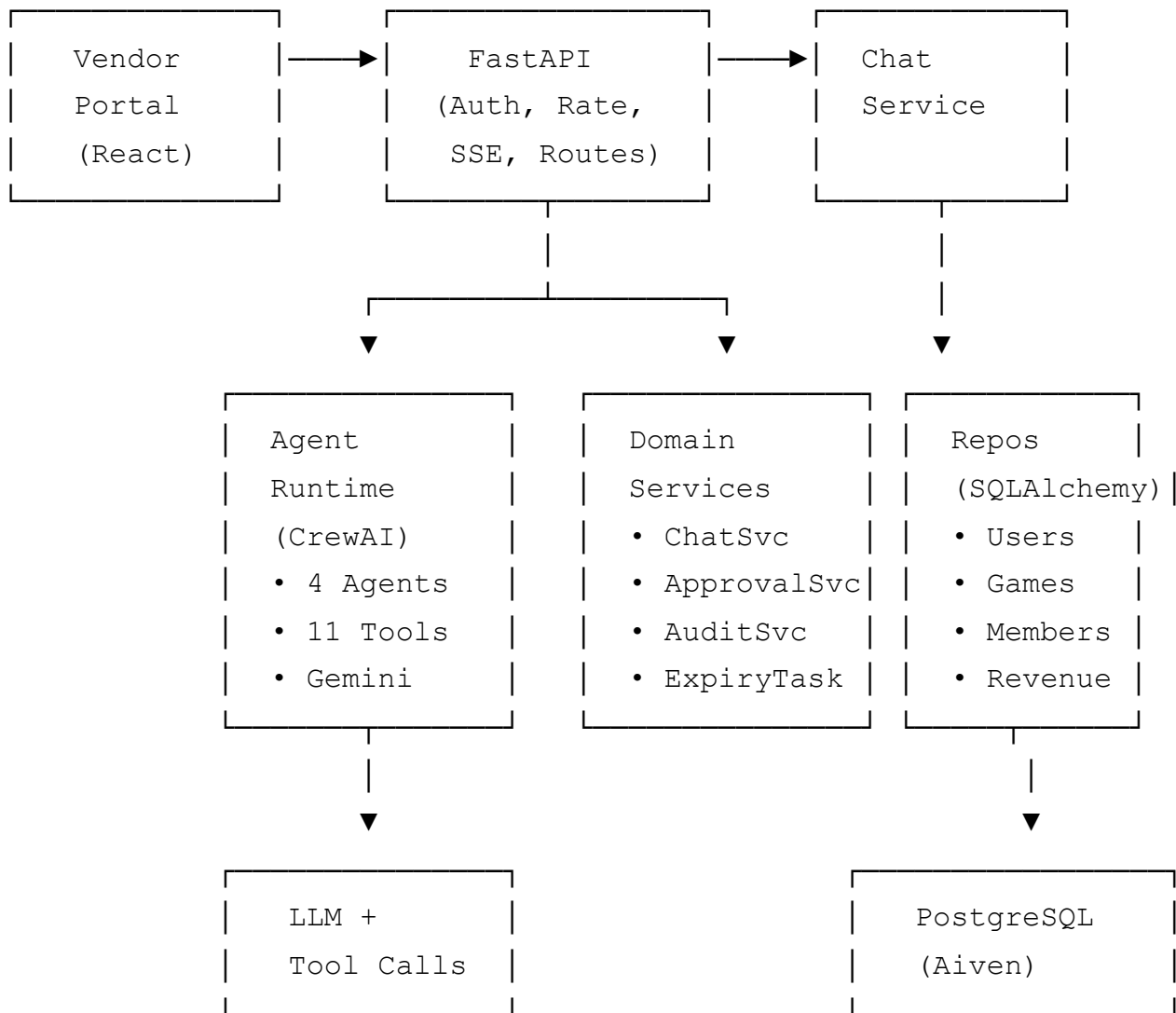
- Composer returns polite refusal with reason

Streaming (SSE)

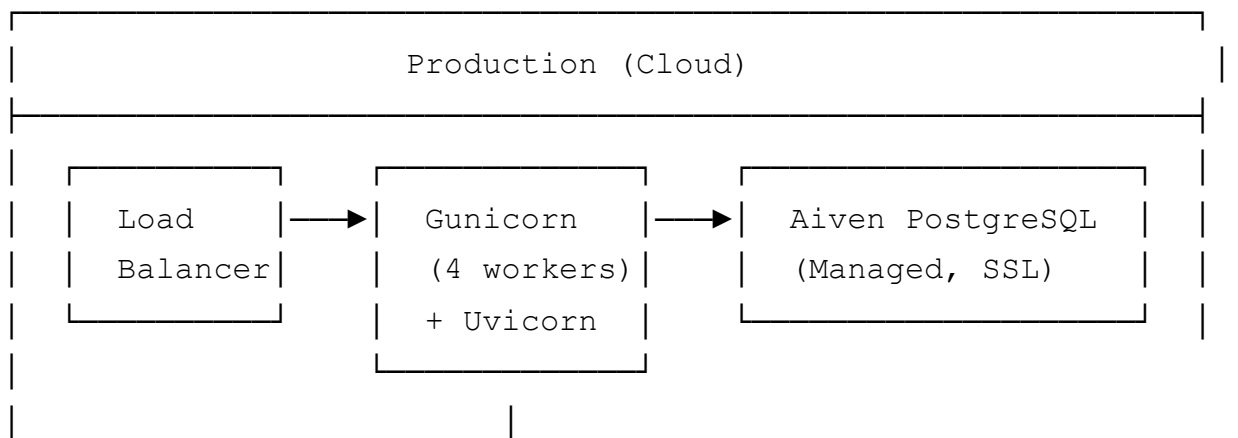
```
# /sessions/{id}/messages:stream
event: status      data: {"phase": "routing"}
event: status      data: {"phase": "running"}
```

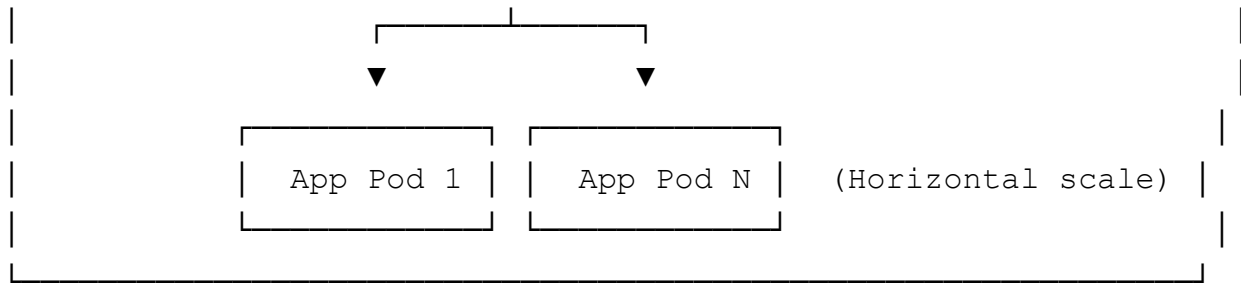
```
event: result      data: {"text": "...", "blocks": [...], "tool_traces": [
event: done        data: {"message_id": "msg_xxx", "session_id": "sess_yyy"}
```

5. Data Flow Summary



6. Deployment Architecture





Key Production Configurations

Component	Configuration
Workers	<code>gunicorn -w 4 -k uvicorn.workers.UvicornWorker</code>
Health Check	<code>/v1/health</code> (liveness + DB connectivity)
DB Pool	Async pool (asyncpg) + Sync pool (psycopg2)
Background Tasks	Proposal expiry → Cron job (<code>run_expiry_once()</code>)
Secrets	<code>.env</code> → Docker secrets / cloud secret manager
Logs	Structured JSON → Cloud logging (CloudWatch, Datadog, etc.)

7. Decision Log

Decision	Rationale	Alternatives Considered
CrewAI over LangGraph	Simpler for fixed sequential pipeline; built-in tool calling	LangGraph (more flexible but complex)
Gemini 3.1 Flash-Lite	Low cost, fast, native tool calling, structured output	GPT-4o-mini (higher cost), Claude Haiku (no native tools in LangChain)
Propose→Approve→Execute	Regulatory compliance; human-in-the-loop safety	Direct tool execution (rejected)
Header-based auth (Phase 0-3)	Rapid prototyping; JWT/OIDC adds complexity	JWT from start (deferred to Phase 4)
SQLite dev / PostgreSQL prod	Zero-config local dev; production parity via asyncpg	PostgreSQL everywhere (adds local complexity)
Session-scoped event loop	Fixes asyncpg "Event loop closed" in pytest	Function-scoped (caused deadlocks)

8. Current Status & Next Steps

Phase	Status	Tests
0: Foundation	✔ Complete	12/12 pass
1: Read Copilot	✔ Complete	15/15 pass
2: Write + Approval	✔ Complete	11/11 pass
3: Hardening	✔ Complete	42/42 pass
Total	✔ 85/85 pass	

Immediate Next Steps (Phase 4)

- 1. **JWT/OIDC Authentication** – Replace header stubs
- 2. **Output Guardrails** – Validate Composer JSON schema
- 3. **Observability** – OpenTelemetry + distributed tracing
- 4. **Multi-approver workflows** – Sequential/parallel approval chains
- 5. **Bulk operations** – Propose multiple changes in one go
- 6. **Semantic memory** – pgvector for historical Q&A retrieval

Report generated: 2026-07-13
Project: HobbyFi Vendor Portal AI Copilot
Environment: Python 3.13, FastAPI, CrewAI, PostgreSQL (Aiven)